

Lab2 Report

Name: 박수현

Student ID: 2022063672

Class Code: 12916

1. Overall Structure

The highest module in this structure is the TOP module. The TOP module is a simple module that connects two submodules, ALU and Register File. ALU is an arithmetic logic unit that performs arithmetic and logic operations on data. The Register File is a temporary storage that stores operands and results for the ALU. TOP module has 8 inputs, 6 for RF and 2 for ALU. The two operands for ALU are connected from read data of RF and write data for RF is connected from the result of ALU.

2. Implementation

The implementation of TOP module and ALU is nothing special to mention. The implementation of ALU module is the same as previous homework and the TOP module is a simple module that simply connects RF and ALU.

The implementation of the register file in this homework is quite simple because I don't have to handle the exceptional cases occurring in pipelining such as forwarding. The task performed by RF can be divided into read and write. Read operation is reading a value stored in register of given register number, rd_addr. This operation happens asynchronously; the rd_data changes as soon as rd_addr changes. So, this operation should be implemented in always(*) block. Write operation change the value of the register file to wr_data. This operation happens synchronously; wr_data is written to the register file at the positive edge of the clock. So, this operation should be implemented in always @(posedge clk) block.

```
20
21 // Fill in the asynchronous functions - read
22 always @(*) begin
23     // if(RegWrite && rd_addr1 == wr_addr)
24     // rd_data1 = wr_data;
25     // else
26     rd_data1 = register_file[rd_addr1];
27
28
29     // if(RegWrite && rd_addr2 == wr_addr)
30     // rd_data2 = wr_data;
31     // else
32     rd_data2 = register_file[rd_addr2];
33
34 end
35 // assign ??
36
37 //synchronous - write
38 always @(posedge clk) begin
39     if (rst) begin
40         $readmemh("initial_reg.mem", register_file);
41     end
42
43     // FILL what happens synchronously
44     else if(RegWrite) register_file[wr_addr] <= wr_data;
45 end
```

3. Troubleshooting

Initially, I tried to implement forwarding techniques I just learned in the last lecture. However, it doesn't work. That is because forwarding is only used when reading and writing to the same register in the same cycle. But, in this case, `wr_data` is not yet calculated because the output value of RF is not assigned to the operands of ALU in that period. So, the value of `wr_data` is not the data I wanted. After realizing that issue, I removed all forwarding codes, and it passes all test cases.